

CBM Mobile Application

Technical Documentation

Version: 1.2.1

Date: 12-02-2026

Prepared By: Development Team (Pavan A)

Organization: INDXO AI PVT Ltd.

Table of Contents

- [1. Project Overview](#)
- [2. Technology Stack](#)
- [3. System Architecture](#)
- [4. Application Navigation Flow](#)
- [5. Module Documentation](#)
- [6. Screen Documentation](#)
- [7. Data Flow](#)
- [8. API Integration](#)
- [9. Security](#)
- [10. Device Features Used](#)
- [11. Error Handling](#)
- [12. Future Enhancements](#)

1. Project Overview

1.1 Purpose of the Application

The CBM Mobile Application is a comprehensive field operations management system designed for CBM India Trust. The application enables field workers to collect, manage, and submit data related to various healthcare and community development programs. It supports

multiple modules including door-to-door surveys, vision screening, education programs, humanitarian assistance, community outreach, and training attendance tracking.

The application is built with an offline-first architecture, allowing field workers to continue data collection even in areas with limited or no internet connectivity. Data is automatically synchronized when connectivity is restored.

1.2 Target Users

- **Field Workers:** Primary users who collect data in the field, conduct surveys, and manage patient information
- **Administrators:** Users who manage projects, assign tasks, view reports, and manage user credentials
- **Healthcare Professionals:** Users conducting vision screenings, medical assessments, and referrals
- **Program Coordinators:** Users managing community programs, training sessions, and awareness campaigns

1.3 Key Features

- **Multi-Module Data Collection:** Support for door surveys, vision screening, education questionnaires, community programs, and humanitarian assistance
- **Offline Capability:** Complete offline functionality with automatic data synchronization
- **GPS Location Tracking:** Automatic geotagging of all data submissions
- **Image Capture:** Camera integration for capturing photos, documents, and signatures
- **Digital Signatures:** Electronic signature capture for consent forms
- **Attendance Management:** Clock-in/clock-out functionality with location tracking
- **Task Management:** Daily task assignment and follow-up tracking
- **User Profile Management:** Profile updates, password changes, and image uploads
- **Admin Dashboard:** Comprehensive admin panel for project management and reporting

2. Technology Stack

2.1 Frontend

Technology	Version/Purpose
React Native	0.79.5 - Cross-platform mobile framework
Expo	~53.0.22 - Development platform and tooling
TypeScript	~5.8.3 - Type-safe JavaScript
Expo Router	~5.1.7 - File-based routing system
React Navigation	^7.1.6 - Navigation library
AsyncStorage	2.1.2 - Local data persistence
Expo SQLite	~15.2.14 - Offline database

2.2 Backend

Technology	Version/Purpose
Node.js	Runtime environment
Express.js	^5.1.0 - Web framework
JSON Web Token (JWT)	^9.0.2 - Authentication
bcryptjs	^3.0.2 - Password hashing
mssql	^11.0.1 - SQL Server database driver

2.3 Database

- **SQL Server:** Primary database for storing all application data
- **SQLite:** Local offline database for mobile devices

2.4 Mobile Features Used

Feature	Library/Package	Purpose
Camera	expo-camera, expo-image-picker	Capture photos, documents, and selfies
GPS Location	expo-location	Geotagging of data submissions

File System	expo-file-system	File storage and management
Document Picker	expo-document-picker	Select documents from device
Signature Capture	react-native-signature-canvas	Electronic signature capture
Maps	expo-maps, react-native-maps	Location visualization
Network Info	@react-native-community/netinfo	Network connectivity detection

3. System Architecture

3.1 Architecture Overview

The CBM Mobile Application follows a three-tier architecture:

Mobile Application Layer

- React Native UI Components
- Expo Router Navigation
- Local SQLite Database (Offline Storage)
- AsyncStorage (User Preferences & Tokens)
- Offline Queue Manager

API Layer

- RESTful API Endpoints
- JWT Authentication Middleware
- Request/Response Handling
- Error Handling & Validation

Backend Server

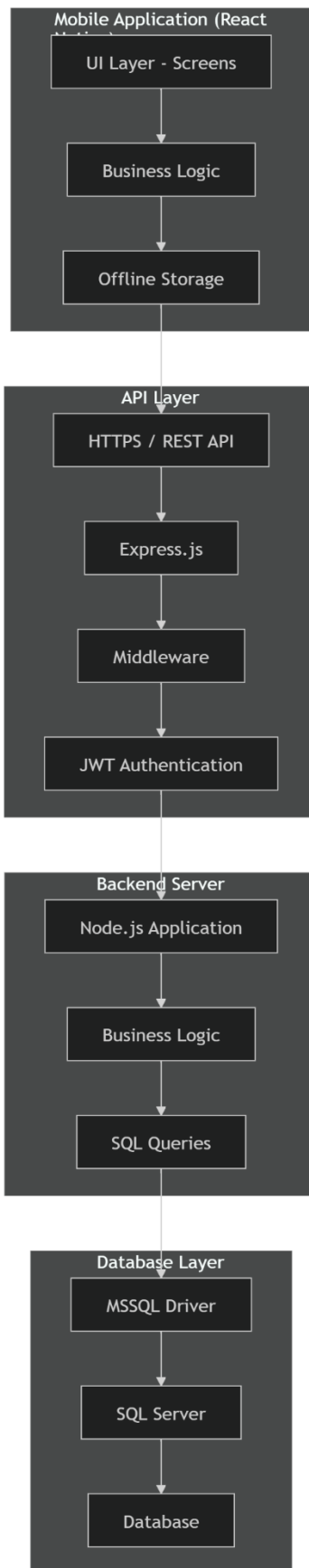
- Node.js/Express.js Server
- Business Logic Processing

- Data Validation
- Database Connection Pooling

Database Layer

- SQL Server Database
- Stored Procedures
- Data Tables for all modules

3.2 Architecture Diagram



3.3 Data Flow Architecture

The application implements an offline-first architecture:

1. **Online Mode:** Data is submitted directly to the API and stored in SQL Server
2. **Offline Mode:** Data is stored in local SQLite database and queued for synchronization
3. **Sync Manager:** Automatically syncs queued data when connectivity is restored

4. Application Navigation Flow

4.1 Login Flow

1. User opens application → Redirected to Login Screen
2. User enters username and password
3. Application checks for existing valid token (Auto-login)
4. If no token, sends credentials to API
5. Backend validates credentials and returns JWT token
6. Token stored in AsyncStorage
7. User redirected based on role:
 - Admin → Admin Dashboard
 - Regular User → User Dashboard

4.2 Dashboard Flow

1. User lands on Dashboard after login
2. Dashboard displays:
 - Banner carousel with images
 - Project selection dropdown
 - Activity selection dropdown
 - Quick action buttons for assigned sub-activities
 - Today's follow-up tasks
3. User selects Project and Activity
4. System fetches assigned sub-activities
5. Quick action buttons appear for available modules
6. User clicks module button → Navigates to consent screen or module screen

4.3 Module Navigation Flow

Door Survey Module:

1. Dashboard → Door to Door Quick Action
2. Door Consent Screen (if consent not given)
3. Door Survey Form (Form 1: Household, Form 2: Members)

4. Data Submission → Dashboard

Vision Screening Module:

1. Dashboard → Vision Center Quick Action
2. Vision Consent Screen
3. Vision Screening Form (Patient Details, Screening, Refraction, Spectacles)
4. Data Submission → Dashboard

Education Module:

1. Dashboard → Education Quick Action
2. Education Consent Screen
3. Development Questions Screen
4. Data Submission → Dashboard

4.4 Menu Navigation

The application includes a custom drawer menu accessible from most screens:

- **Home:** Returns to Dashboard
- **Attendance:** Opens Attendance screen
- **Education:** Opens Education module
- **Profile:** Opens User Profile screen
- **Logout:** Clears session and returns to Login

5. Module Documentation

5.1 Door Survey Module

Purpose

Conducts comprehensive door-to-door household surveys to collect information about household members, their health conditions, disabilities, and service requirements.

Screens Used

- **doorconsent.tsx:** Consent form with digital signature
- **door.tsx:** Main survey form with two sections:
 - Form 1: Household information, head of household details, referrals, complaints
 - Form 2: Individual member details, disabilities, complaints, referrals

Data Captured

- Household information (address, GPS coordinates, contact details)
- Head of household details (name, age, gender, disability information)
- Household referrals and chief complaints

- Individual member details (name, age, gender, relationship)
- Member-specific disabilities, complaints, and referrals
- Government ID documents (Aadhaar, etc.)
- UDID card information
- Photos and attachments

API Integration

- **POST /api/door/consent:** Submit consent form
- **POST /api/door/visit:** Submit household visit data
- **POST /api/door/household_referral:** Submit household referrals
- **POST /api/door/household_complaint:** Submit household complaints
- **POST /api/door/household_member:** Submit individual member data
- **POST /api/door/member_complaint:** Submit member complaints
- **POST /api/door/member_referral:** Submit member referrals

5.2 Vision Screening Module

Purpose

Conducts comprehensive vision screening and eye care assessments for patients, including refraction testing, spectacles prescription, and surgical referrals.

Screens Used

- **visionconsent.tsx:** Patient consent form
- **vision.tsx:** Complete vision screening form with multiple sections

Data Captured

- Patient demographics (MR ID, name, DOB, age, gender, contact)
- Address and village information
- Disability information and UDID card details
- Chief complaints
- Refraction details (right and left eye)
- Prescribed and dispensed spectacles
- Frame cost information
- Surgery willingness and travel arrangements
- Blood pressure and sugar readings
- Physical fitness assessment
- Doctor screening notes

API Integration

- **POST /api/vision/register:** Register patient
- **POST /api/vision/chief_complaints:** Submit complaints

- **POST /api/vision/screening:** Submit screening data
- **POST /api/vision/refraction_details:** Submit refraction details
- **POST /api/vision/refraction:** Submit refraction data
- **POST /api/vision/spectacles:** Submit spectacles information
- **POST /api/vision/cataract:** Submit cataract information
- **POST /api/vision/eye_exam:** Submit eye examination data
- **POST /api/vision/referrals:** Submit referral information

5.3 Education Module

Purpose

Collects educational assessment data through development questionnaires to evaluate learning capabilities and identify support needs.

Screens Used

- **educationconcent.tsx:** Education consent form
- **education.tsx:** Education form
- **DevelopmentQuestionsScreen.tsx:** Development assessment questionnaire

Data Captured

- Student/participant basic information
- Educational background
- Development assessment responses
- Learning difficulties identification
- Support requirements

API Integration

- **POST /api/education/consent:** Submit education consent
- **POST /api/education/register:** Register education participant
- **POST /api/education/assessment:** Submit assessment data

5.4 Humanitarian Module

Purpose

Manages humanitarian assistance programs, including family registration, item distribution tracking, and geotagged photo documentation.

Screens Used

- **humanitarien/dashboard.tsx:** Humanitarian module dashboard
- **familyform.tsx:** Family registration form
- **GeoTaggedCameraScreen.tsx:** Camera screen with GPS tagging

Data Captured

- Family information and demographics
- Family member details
- Item distribution records
- Geotagged photographs
- Location coordinates

API Integration

- **POST /api/humanitarian/family/create:** Create family entry
- **POST /api/humanitarian/family/upload-image:** Upload family photos
- **GET /api/humanitarian/dashboard-summary:** Get dashboard statistics
- **GET /api/humanitarian/items:** Get available items

5.5 Community Program Module

Purpose

Manages community outreach programs and camp activities, collecting participant information and program outcomes.

Screens Used

- **communityconsent.tsx:** Community program consent
- **community.tsx:** Community program form

Data Captured

- Program/camp details
- Participant information
- Services provided
- Referrals made
- Outcomes and follow-up requirements

API Integration

- **POST /api/community/consent:** Submit community consent
- **POST /api/community/outreach:** Submit outreach data

5.6 Training Attendance Module

Purpose

Tracks attendance for training programs, including participant details, session information, and attendance records.

Screens Used

- **TrainingAttendanceForm.tsx:** Training attendance form
- **cbidtrainingconsent.tsx:** CBID training consent

Data Captured

- Training program details
- Participant information
- Attendance records
- Session dates and times
- Training materials and certificates

API Integration

- **POST /api/training/attendance:** Submit training attendance
- **POST /api/cbid/training:** Submit CBID training data

5.7 School Screening Module

Purpose

Conducts health and vision screenings in schools, collecting student information and screening results.

Screens Used

- **schoolconsent.tsx:** School screening consent
- **school.tsx:** School screening form

Data Captured

- School information
- Student demographics
- Screening results
- Referrals made

API Integration

- **POST /api/school/consent:** Submit school consent
- **POST /api/school/screening:** Submit screening data

6. Screen Documentation

Screen Name	File Path	Purpose	Navigation Entry
Login	app/login.tsx	User authentication and login	Application entry point, auto-redirect from index
Dashboard	app/screens/dashboard.tsx	Main dashboard with project/activity selection and quick actions	After login (regular users), Menu drawer "Home"
Admin Dashboard	app/screens/admin/dashboard.tsx	Administrative dashboard for admin users	After login (admin users)
Attendance	app/screens/attendance.tsx	Clock-in/clock-out functionality with location tracking	Menu drawer "Attendance"
Door Survey	app/screens/door.tsx	Door-to-door	Dashboard → Door to Door quick

		household survey form	action → Door Consent
Door Consent	app/screens/consent/doorconsent.tsx	Consent form for door survey with digital signature	Dashboard → Door to Door quick action
Vision Screening	app/screens/vision.tsx	Complete vision screening and eye care assessment	Dashboard → Vision Center quick action → Vision Consent
Vision Consent	app/screens/consent/visionconsent.tsx	Consent form for vision screening	Dashboard → Vision Center quick action
Education	app/screens/education/education.tsx	Education program form	Dashboard → Education quick action → Education Consent
Development Questions	app/screens/education/DevelopmentQuestionsScreen.tsx	Development assessment questionnaire	From Education form

Education Consent	app/screens/consent/educationconsent.tsx	Consent form for education programs	Dashboard → Education quick action
Community Program	app/screens/community.tsx	Community outreach and camp program form	Dashboard → Community Camp quick action → Community Consent
Community Consent	app/screens/consent/communityconsent.tsx	Consent form for community programs	Dashboard → Community Camp quick action
School Screening	app/screens/school.tsx	School health and vision screening form	Dashboard → School Screening quick action → School Consent
School Consent	app/screens/consent/schoolconsent.tsx	Consent form for school screening	Dashboard → School Screening quick action

Base Hospital	app/screens/basehospital.tsx	Base hospital data collection form	Dashboard → Base Hospital quick action
Training Attendance	app/screens/TrainingAttendanceForm.tsx	Training program attendance tracking	Dashboard → Training Program quick action
Awareness Programme	app/screens/AwarenessProgrammeForm.tsx	Awareness program data collection	Dashboard → Awareness Training quick action
CBID Training	app/screens/cbidtraining.tsx	CBID training data collection	Dashboard → CBID Training quick action
CBID Training Consent	app/screens/consent/cbidtrainingconsent.tsx	Consent form for CBID training	Dashboard → CBID Training quick action
Humanitarian Dashboard	app/screens/humanitarien/dashboard.tsx	Humanitarian module dashboard with statistics	Dashboard → Humanitarian Module

			quick action
Humanitarian Family Form	app/screens/humanitarien/familyform.tsx	Family registration for humanitarian assistance	From Humanitarian Dashboard
GeoTagged Camera	app/screens/humanitarien/GeoTaggedCameraScreen.tsx	Camera screen with GPS location tagging	From Humanitarian Family Form
Profile	app/screens/profile.tsx	User profile management and password change	Menu drawer profile icon, Profile menu item
Eye Care	app/screens/eyecare.tsx	Eye care module navigation hub	Menu navigation
User Credentials (Admin)	app/screens/admin/usercredentials.tsx	Admin screen for managing user credentials	Admin Dashboard → User Credentials menu
Assign Task (Admin)	app/screens/admin/assigntask.tsx	Admin screen for assigning	Admin Dashboard

		tasks to users	→ Assign Task menu
Info Panel (Admin)	app/screens/admin/infopanel.tsx	Admin information panel	Admin Dashboard → Info Panel menu

7. Data Flow

7.1 User Input Flow

1. User opens a form screen (e.g., Door Survey, Vision Screening)
2. User fills in form fields (text inputs, dropdowns, date pickers)
3. User captures images using camera or selects from gallery
4. User captures GPS location (automatic or manual)
5. User captures digital signature (for consent forms)
6. Form data is stored in component state

7.2 Form Validation

1. Client-side validation checks required fields
2. Data type validation (dates, numbers, emails)
3. File size validation (images, documents)
4. Image format validation
5. GPS coordinate validation
6. If validation fails, error messages displayed to user
7. If validation passes, proceed to submission

7.3 API Request Flow (Online Mode)

1. Application checks network connectivity
2. If online, prepares API request payload
3. Retrieves JWT token from AsyncStorage
4. Adds Authorization header with Bearer token
5. Sends HTTP POST/PUT request to API endpoint
6. Backend receives request and validates token
7. Backend processes business logic
8. Backend executes database operations

9. Backend returns response (success/error)
10. Mobile app handles response:
 - Success: Shows confirmation, navigates to dashboard
 - Error: Shows error message, allows retry

7.4 Offline Mode Flow

1. Application detects no network connectivity
2. Data is saved to local SQLite database
3. Data is added to offline queue table
4. User receives confirmation that data is saved offline
5. Sync Manager periodically checks for connectivity
6. When connectivity restored:
 - Sync Manager processes queued items
 - Each item is sent to API
 - On success, item marked as synced
 - On failure, item remains in queue for retry

7.5 Backend Processing

1. Express.js receives HTTP request
2. JWT middleware validates token
3. Request body parsed and validated
4. Business logic executed
5. SQL Server connection established
6. Database queries executed (INSERT/UPDATE/SELECT)
7. Stored procedures called if needed
8. Transaction committed
9. Response sent back to mobile app

7.6 Database Storage

Data is stored in SQL Server database with the following structure:

- **User Tables:** User master, user profiles, authentication
- **Project Tables:** Projects, activities, sub-activities
- **Module Tables:** Separate tables for each module (door visits, vision screenings, etc.)
- **Reference Tables:** Referrals, complaints, attachments
- **System Tables:** Attendance, tasks, offline queue

8. API Integration

8.1 API Base URL

Base URL: http://103.25.126.212:8080

All API endpoints are prefixed with /api

8.2 Authentication

All API requests (except login) require JWT authentication:

```
Authorization: Bearer {JWT_TOKEN}
```

8.3 API Endpoints

Authentication Endpoints

Method	Endpoint	Purpose
POST	/api/auth/login	User login, returns JWT token
GET	/api/user/profile	Get user profile information
POST	/api/user/profile/image	Update user profile image
POST	/api/user/profile/password	Change user password

Dashboard Endpoints

Method	Endpoint	Purpose
GET	/api/user/projects	Get list of projects assigned to user

GET	/api/user/projects/:projectId/activities	Get activities for a project
GET	/api/user/projects/:projectId/activities/:activityId/subactivities	Get sub-activities for an activity
GET	/api/user/dailytasks	Get daily tasks and follow-ups
PUT	/api/referral/update/:type/:taskId	Update referral task status or date

Door Survey Endpoints

Method	Endpoint	Purpose
POST	/api/door/consent	Submit door survey consent
POST	/api/door/visit	Submit household visit data
POST	/api/door/household_referral	Submit household referrals
POST	/api/door/household_complaint	Submit household complaints

POST	/api/door/household_member	Submit household member data
POST	/api/door/member_complaint	Submit member complaints
POST	/api/door/member_referral	Submit member referrals

Vision Screening Endpoints

Method	Endpoint	Purpose
POST	/api/vision/register	Register vision screening patient
POST	/api/vision/chief_complaints	Submit patient complaints
POST	/api/vision/screening	Submit screening data
POST	/api/vision/refraction_details	Submit refraction details
POST	/api/vision/refraction	Submit refraction data
POST	/api/vision/spectacles	Submit spectacles information
POST	/api/vision/cataract	Submit cataract information
POST	/api/vision/eye_exam	Submit eye examination data

POST	/api/vision/referrals	Submit referral information
------	-----------------------	-----------------------------

Attendance Endpoints

Method	Endpoint	Purpose
POST	/api/attendance/checkin	Clock in with location
POST	/api/attendance/checkout	Clock out
GET	/api/attendance/today	Get today's attendance record

Other Module Endpoints

- **Community:** /api/community/consent, /api/community/outreach
- **School:** /api/school/consent, /api/school/screening
- **Education:** /api/education/consent, /api/education/register
- **Training:** /api/training/attendance, /api/cbid/training
- **Humanitarian:** /api/humanitarian/family/create, /api/humanitarian/family/upload-image
- **Awareness:** /api/awareness/programme

8.4 Request/Response Format

Request Headers:

```
Content-Type: application/json Authorization: Bearer {JWT_TOKEN}
```

Request Body: JSON format

Response Format: JSON

- **Success:** { "success": true, "data": {...}, "id": 123 }
- **Error:** { "error": "Error message", "code": "ERROR_CODE" }

9. Security

9.1 Authentication Mechanism

The application uses JSON Web Tokens (JWT) for authentication:

Login Process

1. User enters username and password
2. Credentials sent to `/api/auth/login`
3. Backend validates credentials against SQL Server database
4. Backend checks password using bcrypt or stored procedure
5. If valid, backend generates JWT token with payload:
 - `user_id`
 - `username`
 - `email`
 - `role (admin/user)`
6. Token signed with secret key
7. Token returned to mobile app
8. Token stored in AsyncStorage
9. Token expiry set to 30 days

9.2 Token Storage

- JWT token stored in AsyncStorage with key "token"
- User information stored in AsyncStorage with key "user"
- Login timestamp and expiry stored for session management
- Token automatically included in all API requests via Authorization header

9.3 Token Validation

Backend validates tokens on every protected API request:

1. Extracts token from Authorization header
2. Verifies token signature using secret key
3. Checks token expiry
4. If valid, extracts user information and attaches to request object
5. If invalid, returns 401 Unauthorized error

9.4 Auto-Login

The application implements auto-login functionality:

- On app launch, checks for existing valid token in AsyncStorage
- If token exists and not expired, automatically logs user in
- User redirected to appropriate dashboard based on role
- If token expired or invalid, user redirected to login screen

9.5 Offline Login

The application supports offline login for previously authenticated users:

- User credentials (hashed) stored in local SQLite database
- If online login fails due to network error, attempts offline login
- Offline login validates against local database
- Allows user to continue working offline

9.6 Password Security

- Passwords hashed using bcrypt before storage
- Password change requires current password verification
- New passwords validated for strength
- Password updates require authentication

9.7 Data Security

- All API communications use HTTPS (in production)
- Sensitive data encrypted in transit
- Local database (SQLite) stored securely on device
- Images and documents stored as base64 in database
- GPS coordinates included for data verification

10. Device Features Used

10.1 Camera

Purpose

- Capture photos of patients/participants
- Document government IDs (Aadhaar, etc.)
- Capture selfies for verification
- Document physical conditions
- Geotagged photos for humanitarian module

Implementation

- **Library:** expo-camera, expo-image-picker
- Camera permissions requested on first use
- Images compressed before upload
- Base64 encoding for API transmission
- File size validation (max 5MB)

10.2 GPS Location

Purpose

- Geotag all data submissions
- Track field worker locations

- Verify data collection locations
- Map visualization of data points
- Attendance location tracking

Implementation

- **Library:** expo-location
- Location permissions requested on first use
- Coordinates captured automatically on form submission
- Latitude and longitude stored with each record
- Location accuracy settings configured

10.3 Image Upload

Purpose

- Upload profile images
- Upload document attachments
- Upload consent form signatures
- Upload government ID documents

Implementation

- Images selected from gallery or captured via camera
- Images converted to base64 format
- File size validation and compression
- Multiple image uploads supported
- Progress indicators for uploads

10.4 Signature Capture

Purpose

- Digital signatures on consent forms
- Patient/participant consent verification
- Legal documentation

Implementation

- **Library:** react-native-signature-canvas
- Canvas-based signature drawing
- Signatures converted to base64 PNG
- Stored as binary data in database
- Displayed on forms for verification

10.5 File System

Purpose

- Store offline data
- Cache images and documents
- Manage local database files

Implementation

- **Library:** expo-file-system
- SQLite database file management
- Temporary file storage
- File cleanup and maintenance

10.6 Network Detection

Purpose

- Detect online/offline status
- Switch between online and offline modes
- Trigger automatic synchronization

Implementation

- **Library:** @react-native-community/netinfo
- Continuous network monitoring
- Connection state change listeners
- Automatic sync trigger on reconnection

11. Error Handling

11.1 API Error Handling

Network Errors

- Timeout errors detected and handled
- Connection failures trigger offline mode
- Data automatically saved to offline queue
- User notified that data will sync when online

HTTP Error Responses

- **401 Unauthorized:** Token expired or invalid → Redirect to login
- **400 Bad Request:** Validation errors → Display error message to user
- **500 Server Error:** Server-side errors → Log error, show generic message
- **404 Not Found:** Endpoint not found → Log error for debugging

11.2 Validation Error Handling

- Client-side validation before API submission
- Required field validation
- Data type validation (dates, numbers)
- File size and format validation
- Error messages displayed inline with form fields
- Form submission blocked until validation passes

11.3 Offline Error Handling

- Network failures automatically trigger offline mode
- Data saved to local SQLite database
- Failed requests added to offline queue
- Queue processed automatically when connectivity restored
- Retry mechanism for failed syncs
- User notified of sync status

11.4 Database Error Handling

- SQLite errors caught and logged
- Transaction rollback on errors
- Database connection errors handled gracefully
- Data integrity maintained through transactions

11.5 User-Friendly Error Messages

- Technical errors translated to user-friendly messages
- Actionable error messages (e.g., "Please check your internet connection")
- Retry options provided where applicable
- Error logging for debugging purposes

11.6 Error Recovery

- Automatic retry for transient network errors
- Manual retry option for users
- Data persistence ensures no data loss
- Session recovery on app restart

12. Future Enhancements

12.1 Enhanced Offline Mode

- Complete offline functionality for all modules
- Offline data editing and updates
- Conflict resolution for data synchronization

- Offline reporting and analytics
- Data compression for efficient storage

12.2 Data Synchronization Improvements

- Incremental sync to reduce data transfer
- Background synchronization
- Sync progress indicators
- Selective sync (sync only changed data)
- Sync conflict resolution UI

12.3 Admin Analytics Dashboard

- Real-time data visualization
- Charts and graphs for key metrics
- Geographic data visualization on maps
- Export capabilities (PDF, Excel)
- Custom report generation
- Trend analysis and forecasting

12.4 Reporting System

- Automated report generation
- Scheduled report delivery
- Multiple report formats (PDF, Excel, CSV)
- Custom report builder
- Report sharing and distribution
- Historical report archiving

12.5 Additional Features

- **Push Notifications:** Task reminders, sync status, important updates
- **Multi-language Support:** Localization for regional languages
- **Biometric Authentication:** Fingerprint/Face ID login
- **Data Export:** Export data to external formats
- **Advanced Search:** Full-text search across all modules
- **Barcode/QR Code Scanning:** Quick data entry via scanning
- **Voice Input:** Voice-to-text for form filling
- **Image OCR:** Extract text from document images
- **Real-time Collaboration:** Multi-user data entry
- **Workflow Management:** Automated workflow processes

12.6 Performance Improvements

- Image optimization and lazy loading
- Database query optimization
- API response caching
- Reduced app bundle size
- Faster app startup time

12.7 Security Enhancements

- End-to-end encryption for sensitive data
- Certificate pinning for API communications
- Enhanced biometric authentication
- Audit logging for all data operations
- Role-based access control (RBAC) improvements

12.8 User Experience Improvements

- Improved UI/UX design
- Dark mode support
- Accessibility improvements (screen readers, larger fonts)
- Form auto-save functionality
- Improved navigation and search